UK Stock Intelligence

Codebase Reference Guide

Project	uk-stock-intelligence
Purpose	AI-powered next-day direction prediction for FTSE 350 stocks
Model	XGBoost + LightGBM ensemble, 55 features, Platt calibrated
Data	Yahoo Finance (free), FinBERT (local), FRED macro instruments
Stack	Python 3.14 · FastAPI · SQLite · APScheduler · ReportLab
GitHub	github.com/thetridentgamestudio-ux/uk-stock-intelligence
Total files	45 source files · ~5,200 lines of code

This document lists every source file, configuration file, and script in the project with its location, type, and purpose. Use it as a quick reference when navigating the codebase.

1. Architecture Overview

The application is structured as a three-layer system:

Layer	Technology	Responsibility
ML Engine	XGBoost + LightGBM	55-feature ensemble model predicting next-day direction
Backend API	FastAPI + SQLite	REST endpoints, scheduling, data persistence
Frontend	HTML + Tailwind CSS	Dashboard — predictions, accuracy, market regime

Daily automated schedule:

Time	Job	What it does
08:00 Mon-Fri	fetch_news_sentiment.py	Scrapes BBC/Guardian/CityAM RSS, scores headlines with FinBERT
17:15 Mon-Fri	run_daily_pipeline.py	Fetches closing prices, evaluates accuracy, saves new predictions
Mon 07:00	crontab pmset	Disables Mac sleep for the trading week
Fri 22:00	crontab pmset	Re-enables Mac sleep for the weekend

2. Backend — Core Application

2.1 Entry Points & Configuration

File	Type	Purpose
`backend/app/main.py`	FastAPI app	Creates the FastAPI app, registers all API routers (stocks, predictions, accuracy), adds C
`backend/app/config.py`	Settings	Loads .env file using pydantic-settings. Defines DATABASE_URL, ANTHROPIC_API_KE
`backend/app/database.py`	DB engine	Creates the SQLAlchemy engine for SQLite. Sets check_same_thread=False for SQLite

2.2 Database Models

File	Type	Purpose
`backend/app/models/db_models.py`	ORM models	Defines 4 SQLAlchemy tables: Stock (ticker metadata), StockPrice (OHLCV daily rows),
`backend/app/models/schemas.py`	Pydantic schemas	Request/response validation for the API. PredictionOut includes all 12 fields returned per

3. Machine Learning Engine

3.1 Feature Engineering

All 55 features are computed from raw OHLCV data — no paid data required.

File	Type	Purpose
`backend/app/ml/features.py`	Feature pipeline	Core ML feature computation. 411 lines. Computes all 55 features across 5 groups: (1) B

Feature summary table:

Group	Count	Key Features	Signal Type
Base technical	14	RSI, MACD, Bollinger, MA ratios, volume	Momentum / oscillators
Long Build-Up (LBU)	4	lbu_score (0-4), consecutive up-closes, HHHL, volume trend	Accumulation
Breakout	4	Dist from 20d/52w high, above prev day/week high	Breakout
Golden cross	3	MA20/50, MA50/200 ratios, volume spike	Trend
Extended momentum	3	60d, 120d, 252d returns	Long-term momentum
Candle structure	5	Gap%, body ratio, upper/lower wick	Intraday conviction
Indicator velocity	4	RSI delta 5d, MACD hist delta, BB delta, OBV slope	Signal acceleration
New indicators	9	ATR ratio, ADX, +/-DI, Stoch %K/%D, Williams %R, CMF, CCI, plus accel	Oscillators
Liquidity	3	Amihud 20d, Amihud change 5d, Roll spread	Market microstructure
Calendar	3	Day of week, turn-of-month, month of year	Seasonal effects
Cross-sectional	4	1m/3m/6m/12-1m rank vs FTSE 350	Relative momentum
TOTAL	**55**		

3.2 Model Training

File	Type	Purpose
`backend/app/ml/train.py`	Training pipeline	328 lines. Loads all OHLCV from DB, computes features for all 350 stocks, adds cross-s

Model performance history:

Version	Features	Backtest	Real-world accuracy
v1 — Original	25	50.2%	59.5% (37 predictions, old data)
v2 — Phase-1 signals	25	50.4%	—
v3 — Phase-2 + CS ranks	49	50.6%	52% (344 predictions, Day 1)
v4 — LGB ensemble	49	50.8%	52% (344 predictions, Day 2)
v5 — Phase-3 + calibration	**55**	**54.5%**	**Tracking (current)**

4. Backend — Services

4.1 Data & Prediction Services

File	Type	Purpose
backend/app/services/data_fetcher.py	Data layer	Loads FTSE_STOCKS dict from stocks.json (350 tickers). fetch_prices() calls yfinance w
backend/app/services/predictor.py	Prediction engine	319 lines. Core prediction loop: loads model bundle, runs compute_features() per stock,
backend/app/services/accuracy_checker.py	Accuracy tracking	Queries predictions with was_correct IS NULL and target_date <= today. Fetches price_

4.2 Market Intelligence Services

File	Type	Purpose
backend/app/services/market_sentiment.py	Regime detection	Fetches ^VIX (5 trading days) and ^FTSE (60 calendar days) from Yahoo Finance. Comp
backend/app/services/macro_features.py	Global macro	191 lines. Fetches 5 daily instruments: GBP/USD, Brent crude, S&P 500, DAX, VIX
backend/app/services/sector_features.py	Sector relative strength	172 lines. Groups 350 stocks by sector (normalises Wikipedia capitalisation inconsistenc
backend/app/services/earnings_fetcher.py	Earnings calendar	Fetches earnings dates from 3 yfinance sources: earnings_dates (needs lxml, 8 quarters
backend/app/services/news_sentiment.py	FinBERT sentiment	264 lines. Scrapes 4 free RSS feeds (BBC Business, Guardian, CityAM, This is Money) u

4.3 Support Services

File	Type	Purpose
backend/app/services/scheduler.py	APScheduler	Registers 2 jobs in FastAPI lifespan: _morning_news at 08:00 Mon-Fri (FinBERT RSS) a
backend/app/services/explainer.py	Claude AI	Calls Anthropic claude-haiku-4-5 API to generate plain-English explanation for a stock pr
backend/app/services/rns_scraper.py	RNS headlines	Scrapes RNS regulatory announcements from LSE website for a given ticker. Used in the

5. Backend — API Routes

Endpoint	Method	Purpose
`GET /health`	—	Health check — returns {status: ok, version}
`GET /api/stocks`	stocks.py	List all 350 FTSE stocks with ticker, name, sector, index
`GET /api/predictions/daily`	predictions.py	Returns top 10 BULLISH + top 10 BEARISH predictions with all flags
`GET /api/predictions/macro`	predictions.py	Global macro backdrop: GBP/USD, Brent, SP500, DAX, VIX, macro_sco
`GET /api/predictions/market-sentiment`	predictions.py	UK market regime: VIX level, FTSE momentum, BULLISH/NEUTRAL/BE
`GET /api/predictions/{ticker}/explain`	predictions.py	Claude AI explanation for a specific stock prediction
`GET /api/accuracy/summary`	accuracy.py	Overall accuracy: total predictions, correct %, days tracked
`GET /api/accuracy/history?limit=N`	accuracy.py	Recent predictions with actual outcomes (max 500 rows)
`GET /api/accuracy/walk-forward`	accuracy.py	5-fold rolling walk-forward accuracy evaluation (takes ~2min)
`GET /api/accuracy/by-stock`	accuracy.py	Per-stock accuracy breakdown, best performers first

6. Scripts

All scripts are run from the project root directory. They use sys.path.insert to import backend modules directly.

File	Run frequency	Purpose
`scripts/run_daily_pipeline.py`	Auto 17:15 Mon-Fri (also manually)	4-step daily pipeline: (1) Fetch today's OHLCV prices for all 350 stocks — uses end=toda
`scripts/fetch_news_sentiment.py`	Auto 08:00 Mon-Fri (also manually)	Runs the FinBERT news sentiment pipeline. Fetches RSS feeds, matches headlines to ti
`scripts/train_model.py`	Monthly (or after changes)	Trains XGBoost + LightGBM ensemble on full historical data. Use --evaluate flag for 5-fo
`scripts/fetch_historical.py`	Once (setup)	Fetches 5+ years of OHLCV history for all 350 stocks from Yahoo Finance. Required bef
`scripts/update_stock_list.py`	Quarterly (index rebalancing)	Scrapes FTSE 100 + FTSE 250 constituents from Wikipedia using BeautifulSoup4. Save
`scripts/update_earnings_cache.py`	Quarterly	Fetches earnings announcement dates for all 350 stocks from yfinance (3 sources: earni
`scripts/init_db.py`	Once (setup)	Creates all database tables from the SQLAlchemy ORM models. Safe to re-run — uses C

7. Frontend

File	Type	Purpose
`frontend/index.html`	Dashboard	441 lines. Single-file dark-themed dashboard. No build step — open directly in browser. S

Dashboard badge reference:

Badge	Colour	Meaning
LBU 4/4 ★	Amber	Long Build-Up score 4/4 — maximum accumulation signal (price up 2 days, volume above avg, positive r
Top 20%	Blue	Stock ranks in top 20% of FTSE 350 by 1-month return (cross-sectional momentum)
Bot 20%	Orange	Stock ranks in bottom 20% — weak relative momentum
■ Earnings in Xd	Yellow	Earnings announcement in X days — confidence reduced (high uncertainty)
■ Post-earnings (Xd)	Yellow	X days after earnings — PEAD (Post-Earnings Announcement Drift) boost applied
■ +ve news (X articles)	Sky blue	FinBERT scored X headlines as positive — confidence boosted for BULLISH picks
■ -ve news (counter-signal)	Sky blue	FinBERT negative sentiment conflicts with BULLISH prediction — confidence reduced
■ Macro tailwind	Purple	GBP/Brent/SP500/VIX backdrop aligned with prediction direction
■ Macro headwind	Purple	Global macro backdrop conflicts with prediction direction

8. Configuration & Data Files

File	Type	Purpose
`.env`	Environment	Live secrets and paths. DATABASE_URL (absolute SQLite path), ANTHROPIC_API_KE
`.env.example`	Template	Safe template showing required variables. Committed to git. Copy to .env and fill in value
`.gitignore`	Git config	Excludes: .env, *.db, models/*.pkl, earnings_cache.json, news_sentiment_cache.json, __
`requirements.txt`	Dependencies	34 Python packages: fastapi, uvicorn, sqlalchemy, pydantic-settings, yfinance, xgboost, l
`backend/app/data/stocks.json`	Stock universe	350 FTSE tickers: {ticker: {name, sector, index}}. Auto-generated by update_stock_list.py
`backend/app/data/earnings_cache.json`	Earnings dates	Quarterly earnings dates per ticker: {ticker: [date_str,...]}. 250 stocks covered. Refreshed
`backend/app/data/news_sentiment_cache.json`	News cache	Today's FinBERT scores: {ticker: {score, label, count, headlines, updated}}. Refreshed da
`models/xgboost_model.pkl`	Model bundle	Serialised joblib bundle: {model (XGBClassifier), lgb_model (LGBMClassifier), calibrator_
`ukstocks.db`	SQLite database	Main database. Tables: stocks, stock_prices, predictions, news_articles. Excluded from g
`docker-compose.yml`	Docker (unused)	Legacy PostgreSQL compose file from initial scaffold. Kept for reference. SQLite is used
`Makefile`	Dev shortcuts	Common commands: make run (start API), make train (train model), make pipeline (run c
`start_server.sh`	Server script	Bash loop that starts uvicorn and restarts it automatically if it crashes. Used by the Launc

9. macOS System Configuration

These files are outside the project directory and manage the server lifecycle and sleep prevention on macOS.

File / Command	Type	Purpose
`~/Library/LaunchAgents/com.ukstock.server.plist`	LaunchAgent	Starts start_server.sh automatically on login. KeepAlive=true means it restarts if the proc
`~/Library/LaunchAgents/com.ukstock.caffeinate.plist`	LaunchAgent	Runs caffeinate -i -s as a permanent background process. -i prevents idle sleep, -s preve
`sudo pmset -a sleep 0`	System setting	Permanently disables automatic sleep at the system level (not process-dependent). Surv
`sudo pmset -a disablesleep 1`	System setting	Nuclear option — completely disables sleep regardless of other settings. Active Mon-Fri
`/etc/sudoers.d/ukstock-pmset`	Sudoers rule	Allows crontab to run sudo pmset without a password prompt. Content: viveksharma ALL
`crontab (user)`	Cron jobs	5 entries: 08:00 Mon-Fri (news), 17:15 Mon-Fri (pipeline), Mon 07:00 (disable sleep), Fri
`sudo pmset repeat wakeorpoweron MTWRF 07:55:00`	Wake schedule	Mac wakes itself at 7:55am Mon-Fri even from a cold sleep state. Run once to set

Management commands quick reference:

Task	Command
Start server manually	`cd /Users/viveksharma/Claude/uk-stock-intelligence && python3 -m uvicorn backend.app.main:app -`
Check server health	`curl http://localhost:8000/health`
Stop server	`lsof -ti:8000 | xargs kill -9`
View server logs	`tail -f /tmp/uk-stock-server.log`
View pipeline logs	`tail -30 /tmp/uk-stock-pipeline.log`
View news logs	`tail -20 /tmp/uk-stock-news.log`
Check sleep setting	`pmset -g | grep sleep`
Disable sleep now	`sudo pmset -a sleep 0 && sudo pmset -a disablesleep 1`
Enable sleep (weekend)	`sudo pmset -a sleep 30 && sudo pmset -a disablesleep 0`
View crontab	`crontab -l`
Reload server LaunchAgent	`launchctl unload ~/Library/LaunchAgents/com.ukstock.server.plist && launchctl load ~/Library/La`

10. Tests

File	Type	Purpose
tests/conftest.py	Fixtures	pytest fixtures: in-memory SQLite DB, test client, sample stock data
tests/test_api.py	API tests	Tests FastAPI endpoints: health check, predictions/daily, accuracy/summary, market-sen
tests/test_features.py	ML tests	Tests compute_features() output shape, column presence, no NaN in required features, t
tests/test_data_fetcher.py	Data tests	Tests fetch_prices() with mocked yfinance, save_prices() upsert logic, FTSE_STOCKS lo

Run tests with:

```
cd /Users/viveksharma/Claude/uk-stock-intelligence && python3 -m pytest tests/ -v
```

11. Complete File Tree

```
uk-stock-intelligence/
■■■ .env ← Live secrets (gitignored)
■■■ .env.example ← Safe template (committed)
■■■ .gitignore
■■■ Makefile ← Dev shortcuts
■■■ README.md ← Setup guide
■■■ requirements.txt ← 34 Python dependencies
■■■ start_server.sh ← Auto-restart server script
■■■ docker-compose.yml ← Legacy (unused)
■■■ ukstocks.db ← SQLite database (gitignored)
■
■■■ models/
■ ■■■ .gitkeep
■ ■■■ xgboost_model.pkl ← Trained model bundle (gitignored)
■
■■■ backend/
■ ■■■ app/
■ ■■■ main.py ← FastAPI app entry point
■ ■■■ config.py ← Settings (pydantic-settings)
■ ■■■ database.py ← SQLAlchemy engine + session
■ ■■■ api/routes/
■ ■ ■■■ predictions.py ← /api/predictions/* endpoints
■ ■ ■■■ accuracy.py ← /api/accuracy/* endpoints
■ ■ ■■■ stocks.py ← /api/stocks endpoint
■ ■■■ ml/
■ ■ ■■■ features.py ← 55-feature computation (411 lines)
■ ■ ■■■ train.py ← XGB+LGB training + calibration
■ ■■■ models/
■ ■ ■■■ db_models.py ← SQLAlchemy ORM tables
■ ■ ■■■ schemas.py ← Pydantic API schemas
■ ■■■ services/
■ ■ ■■■ predictor.py ← Prediction engine (319 lines)
■ ■ ■■■ accuracy_checker.py
■ ■ ■■■ data_fetcher.py ← yfinance OHLCV fetcher
■ ■ ■■■ earnings_fetcher.py
■ ■ ■■■ market_sentiment.py ← VIX + FTSE regime
■ ■ ■■■ macro_features.py ← GBP/Brent/SP500/DAX/VIX
■ ■ ■■■ news_sentiment.py ← FinBERT RSS scorer (264 lines)
■ ■ ■■■ sector_features.py
■ ■ ■■■ scheduler.py ← APScheduler 08:00 + 17:15
■ ■ ■■■ explainer.py ← Claude AI explanations
■ ■ ■■■ rns_scraper.py
■ ■■■ data/
■ ■■■ stocks.json ← 350 FTSE tickers
■ ■■■ earnings_cache.json ← (gitignored)
■ ■■■ news_sentiment_cache.json ← (gitignored)
■
■■■ frontend/
■ ■■■ index.html ← Dashboard (441 lines, no build step)
■
■■■ scripts/
■ ■■■ run_daily_pipeline.py ← Main daily script
■ ■■■ fetch_news_sentiment.py ← FinBERT morning run
■ ■■■ train_model.py ← Train + walk-forward eval
■ ■■■ fetch_historical.py ← One-time 5yr history fetch
■ ■■■ update_stock_list.py ← Refresh FTSE 350 list
■ ■■■ update_earnings_cache.py ← Refresh earnings dates
```

```
■ ■■■ init_db.py ← Create DB tables
■
■■■ tests/
■■■ conftest.py
■■■ test_api.py
■■■ test_features.py
■■■ test_data_fetcher.py
```

12. Enhancement Roadmap

Research-backed improvements ranked by expected accuracy impact. All are free to implement.

Priority	Enhancement	Expected Impact	Difficulty
1	Monthly retraining + recency weighting (recent data weighted 1.5x)	+1 to +3pp	Medium
2	Regime-conditional models (separate XGB+LGB per BULL/NEUTRAL/BEAR)	+2 to +4pp	Medium
3	Confidence threshold + Kelly position sizing (act only on >60% picks)	High (returns)	Easy
4	UK short interest data (shortdata.co.uk — FCA mandatory disclosures)	+0.5 to +1.5pp	Easy
5	Stacking meta-learner (logistic regression on OOF predictions)	+0.5 to +1.5pp	Medium
6	CatBoost as 3rd ensemble member (uncorrelated errors)	+0.3 to +0.8pp	Easy
7	Director buying signal (PDMR/RNS — FCA insider disclosures)	+0.5 to +1pp	Medium
8	SHAP feature pruning — remove ~15 noisy features, keep 35-40	+0.5 to +1pp	Easy
9	Calendar effects: pre-holiday, triple-witching, UK tax year end	+0.2 to +0.5pp	Easy
10	Google Trends (FTSE 100 only, weekly data, pytrends library)	+0.3 to +0.8pp	Medium